

SQL WORKBENCH PRACTISE

By:- DEEPAK
(2501730097)
DBMS

1. Creating Databases and Tables

```
1 • CREATE DATABASE college_management;
2 • USE college_management;
3
4 • CREATE TABLE Student (
5     student_id INT PRIMARY KEY,
6     student_name VARCHAR(50),
7     department_id INT,
8     age INT,
9     FOREIGN KEY (department_id) REFERENCES Department(department_id)
10 );
11
12 • CREATE TABLE Department (
13     department_id INT PRIMARY KEY,
14     department_name VARCHAR(50),
15     location VARCHAR(50)
16 );
17
18 • CREATE TABLE Course (
19     course_id INT PRIMARY KEY,
20     course_name VARCHAR(50),
21     department_id INT,
22     FOREIGN KEY (department_id) REFERENCES Department(department_id)
23 );
24
25 • CREATE TABLE Enrollment (
26     enrollment_id INT PRIMARY KEY,
27     student_id INT,
28     course_id INT,
29     marks INT,
30     FOREIGN KEY (student_id) REFERENCES Student(student_id),
31     FOREIGN KEY (course_id) REFERENCES Course(course_id)
32 );
33
34
35 • CREATE TABLE Employee (
36     emp_id INT PRIMARY KEY,
37     emp_name VARCHAR(50),
38     manager_id INT
39 );
```

2. Inserting Sample Records

2.1 Department

```
INSERT INTO Department VALUES
(1, 'Computer Science', 'Delhi'),
(2, 'Electronics', 'Noida'),
(3, 'Mechanical', 'Gurgaon'),
(4, 'Civil', 'Delhi'),
(5, 'Artificial Intelligence', 'Faridabad'),
(6, 'Mathematics', 'Noida');
```

2.2 Student

```
INSERT INTO Student VALUES
(101, 'Rahul', 1, 20),
(102, 'Priya', 1, 19),
(103, 'Aman', 2, 21),
(104, 'Neha', 3, 20),
(105, 'Karan', NULL, 22),
(106, 'Simran', 5, 19),
(107, 'Arjun', 1, 20);
```

2.3 Course

```
INSERT INTO Course VALUES
(201, 'DBMS', 1),
(202, 'Python', 1),
(203, 'Digital Electronics', 2),
(204, 'Thermodynamics', 3),
(205, 'Machine Learning', 5),
(206, 'Calculus', 6),
(207, 'Data Structures', 1);
```

2.4 Enrollment

```
INSERT INTO Enrollment VALUES
(1, 101, 201, 85),
(2, 101, 202, 78),
(3, 102, 201, 92),
(4, 102, 207, 88),
(5, 103, 203, 35),
(6, 104, 204, 67),
(7, 106, 205, 95),
(8, 107, 201, 72),
(9, 107, 202, 39);
```

2.5 Employee

```
INSERT INTO Employee VALUES
(1, 'Raj', NULL),
(2, 'Amit', 1),
(3, 'Neha', 1),
(4, 'Simran', 2),
(5, 'Karan', 2);
```

3. Basic Join Questions

3.1 Student name with department name

```
SELECT s.student_name, d.department_name
FROM Student s
JOIN Department d
ON s.department_id = d.department_id;
```

Output:-

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
student_name	department_name		
Rahul	Computer Science		
Priya	Computer Science		
Aman	Electronics		
Neha	Mechanical		
Simran	Artificial Intelligence		
Arjun	Computer Science		

3.2 All student and department locations

```
SELECT s.student_name, d.location
FROM Student s
LEFT JOIN Department d
ON s.department_id = d.department_id;
```

Output:-

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
student_name	location		
Rahul	Delhi		
Priya	Delhi		
Aman	Noida		
Neha	Gurgaon		
Karan	NULL		
Simran	Faridabad		
Arjun	Delhi		

3.3 Course with department names

```
SELECT c.course_name, d.department_name
FROM Course c
JOIN Department d
ON c.department_id = d.department_id;
```

Output:-

Result Grid			Filter Rows:	Export:
	course_name	department_name		
▶	DBMS	Computer Science		
	Python	Computer Science		
	Digital Electronics	Electronics		
	Thermodynamics	Mechanical		
	Machine Learning	Artificial Intelligence		
	Calculus	Mathematics		
	Data Structures	Computer Science		

3.4 Student name ,course name and marks

```

SELECT s.student_name, c.course_name, e.marks
FROM Student s
JOIN Enrollment e
ON s.student_id = e.student_id
JOIN Course c
ON e.course_id = c.course_id;

```

Output:-

Result Grid				Filter Rows:	Export:
	student_name	course_name	marks		
▶	Rahul	DBMS	85		
	Rahul	Python	78		
	Priya	DBMS	92		
	Priya	Data Structures	88		
	Aman	Digital Electronics	35		
	Neha	Thermodynamics	67		
	Simran	Machine Learning	95		
	Arjun	DBMS	72		
	Arjun	Python	39		

3.5 All students who are enrolled

```

SELECT DISTINCT s.student_name
FROM Student s
JOIN Enrollment e
ON s.student_id = e.student_id;

```

Output:-

Result Grid		Filter Rows:
	student_name	
▶	Rahul	
	Priya	
	Aman	
	Neha	
	Simran	
	Arjun	

Inner Join Questions

3.6 Student_Name | Department_Name

```
SELECT s.student_name, d.department_name
FROM Student s
INNER JOIN Department d
ON s.department_id = d.department_id;
```

Output:-

Result Grid	Filter Rows:	E
student_name	department_name	
Rahul	Computer Science	
Priya	Computer Science	
Aman	Electronics	
Neha	Mechanical	
Simran	Artificial Intelligence	
Arjun	Computer Science	

3.7 Students belonging to Computer Science

```
SELECT s.*
FROM Student s
INNER JOIN Department d
ON s.department_id = d.department_id
WHERE d.department_name = 'Computer Science';
```

Output:-

Result Grid

Filter Rows:

Export:

	student_id	student_name	department_id	age
▶	101	Rahul	1	20
	102	Priya	1	19
	107	Arjun	1	20

3.8 Student,Course,Marks

```
SELECT s.student_name, c.course_name, e.marks
FROM Student s
INNER JOIN Enrollment e
ON s.student_id = e.student_id
INNER JOIN Course c
ON e.course_id = c.course_id;
```

Output:-

	student_name	course_name	marks
▶	Rahul	DBMS	85
	Rahul	Python	78
	Priya	DBMS	92
	Priya	Data Structures	88
	Aman	Digital Electronics	35
	Neha	Thermodynamics	67
	Simran	Machine Learning	95
	Arjun	DBMS	72
	Arjun	Python	39

3.9 Department-wise Student details

```
SELECT d.department_name, s.student_id, s.student_name, s.age
FROM Department d
INNER JOIN Student s
ON d.department_id = s.department_id
ORDER BY d.department_name;
```

Output:-

	department_name	student_id	student_name	age
▶	Artificial Intelligence	106	Simran	19
	Computer Science	101	Rahul	20
	Computer Science	102	Priya	19
	Computer Science	107	Arjun	20
	Electronics	103	Aman	21
	Mechanical	104	Neha	20

Left Join Questions

3.10 All students including those without department

```
SELECT s.student_name, d.department_name
FROM Student s
LEFT JOIN Department d
ON s.department_id = d.department_id;
```

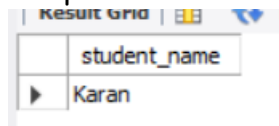
Output:-

	student_name	department_name
▶	Rahul	Computer Science
	Priya	Computer Science
	Aman	Electronics
	Neha	Mechanical
	Karan	NULL
	Simran	Artificial Intelligence
	Arjun	Computer Science

3.11 Students not enrolled in any course

```
SELECT s.student_name
FROM Student s
LEFT JOIN Enrollment e
ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

Output:-

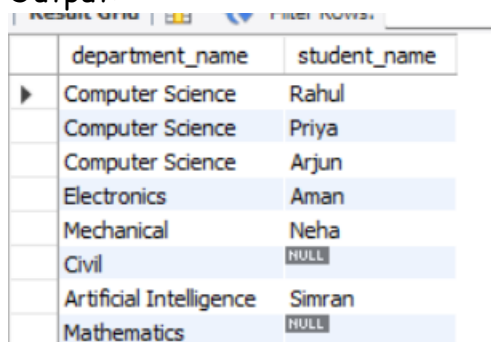


student_name
Karan

3.12 All departments including those with no students

```
SELECT d.department_name, s.student_name
FROM Department d
LEFT JOIN Student s
ON d.department_id = s.department_id;
```

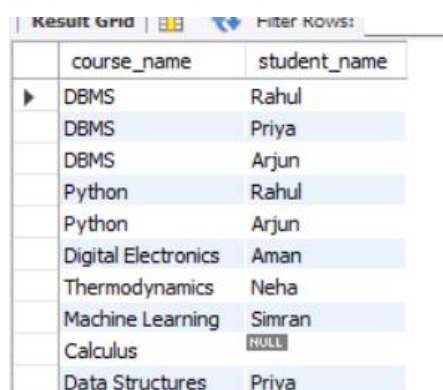
Output:-



department_name	student_name
Computer Science	Rahul
Computer Science	Priya
Computer Science	Arjun
Electronics	Aman
Mechanical	Neha
Civil	NULL
Artificial Intelligence	Simran
Mathematics	NULL

3.13 All courses including courses with no enrolment

```
SELECT c.course_name, s.student_name
FROM Course c
LEFT JOIN Enrollment e
ON c.course_id = e.course_id
LEFT JOIN Student s
ON e.student_id = s.student_id;
```



course_name	student_name
DBMS	Rahul
DBMS	Priya
DBMS	Arjun
Python	Rahul
Python	Arjun
Digital Electronics	Aman
Thermodynamics	Neha
Machine Learning	Simran
Calculus	NULL
Data Structures	Priya

Output:-

Right Join Questions

3.14 All departments and their students

```
SELECT d.department_name, s.student_name
FROM Student s
RIGHT JOIN Department d
ON s.department_id = d.department_id;
```

Output:-

Result Grid	Filter Rows:
department_name	student_name
Computer Science	Rahul
Computer Science	Priya
Computer Science	Arjun
Electronics	Aman
Mechanical	Neha
Civil	NULL
Artificial Intelligence	Simran
Mathematics	NULL

3.15 All courses and enrolled students

```
SELECT c.course_name, s.student_name
FROM Enrollment e
RIGHT JOIN Course c
ON e.course_id = c.course_id
LEFT JOIN Student s
ON e.student_id = s.student_id;
```

Output:-

course_name	student_name
DBMS	Rahul
DBMS	Priya
DBMS	Arjun
Python	Rahul
Python	Arjun
Digital Electronics	Aman
Thermodynamics	Neha
Machine Learning	Simran
Calculus	NULL
Data Structures	Priya

Multiple Join Questions

3.16 Student + Department + Course + Marks

```
SELECT
    s.student_name,
    d.department_name,
    c.course_name,
    e.marks
FROM Student s
JOIN Department d
ON s.department_id = d.department_id
JOIN Enrollment e
ON s.student_id = e.student_id
JOIN Course c
ON e.course_id = c.course_id;
```

Output:-

	student_name	department_name	course_name	marks
▶	Rahul	Computer Science	DBMS	85
	Rahul	Computer Science	Python	78
	Priya	Computer Science	DBMS	92
	Priya	Computer Science	Data Structures	88
	Aman	Electronics	Digital Electronics	35
	Neha	Mechanical	Thermodynamics	67
	Simran	Artificial Intelligence	Machine Learning	95
	Arjun	Computer Science	DBMS	72
	Arjun	Computer Science	Python	39

3.17 Student + Department Location + Course

```
SELECT
    s.student_name,
    d.location,
    c.course_name
FROM Student s
JOIN Department d
ON s.department_id = d.department_id
JOIN Enrollment e
ON s.student_id = e.student_id
JOIN Course c
ON e.course_id = c.course_id;
```

Output:-

Result Grid	Filter Rows:	Ex
student_name	location	course_name
Rahul	Delhi	DBMS
Rahul	Delhi	Python
Priya	Delhi	DBMS
Priya	Delhi	Data Structures
Aman	Noida	Digital Electronics
Neha	Gurgaon	Thermodynamics
Simran	Faridabad	Machine Learning
Arjun	Delhi	DBMS
Arjun	Delhi	Python

3.18 Average marks in each course

```
SELECT
    c.course_name,
    AVG(e.marks) AS average_marks
FROM Course c
JOIN Enrollment e
ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name;
```

Output:-

Result Grid	Filter Rows:
course_name	average_marks
DBMS	83.0000
Python	58.5000
Data Structures	88.0000
Digital Electronics	35.0000
Thermodynamics	67.0000
Machine Learning	95.0000

3.19 Department-wise number of students

```
SELECT
    d.department_name,
    COUNT(s.student_id) AS number_of_students
FROM Department d
LEFT JOIN Student s
ON d.department_id = s.department_id
GROUP BY d.department_id, d.department_name;
```

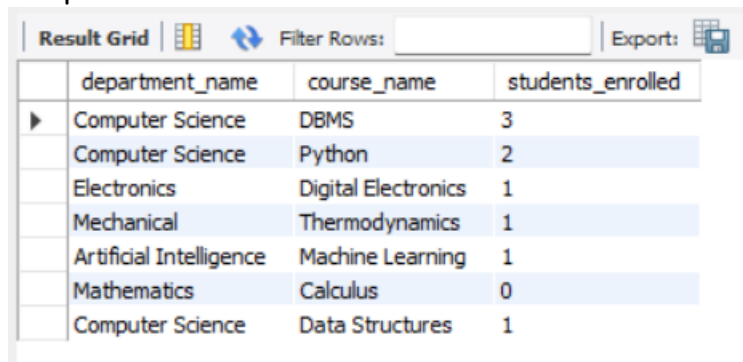
Output:-

Result Grid	Filter Rows:
department_name	number_of_students
Computer Science	3
Electronics	1
Mechanical	1
Civil	0
Artificial Intelligence	1
Mathematics	0

3.20 Department + Course + Number of Students

```
SELECT
    d.department_name,
    c.course_name,
    COUNT(e.student_id) AS students_enrolled
FROM Department d
JOIN Course c
ON d.department_id = c.department_id
LEFT JOIN Enrollment e
ON c.course_id = e.course_id
GROUP BY d.department_id, d.department_name,
         c.course_id, c.course_name;
```

Output:-



The screenshot shows a database interface with a 'Result Grid' tab. It contains a table with 4 columns: department_name, course_name, and students_enrolled. There are 8 rows of data. The interface also includes a 'Filter Rows' search bar and an 'Export' button.

	department_name	course_name	students_enrolled
▶	Computer Science	DBMS	3
	Computer Science	Python	2
	Electronics	Digital Electronics	1
	Mechanical	Thermodynamics	1
	Artificial Intelligence	Machine Learning	1
	Mathematics	Calculus	0
	Computer Science	Data Structures	1

Advanced Join Questions

3.21 Student with highest marks in each course

```
SELECT
    c.course_name,
    s.student_name,
    e.marks
FROM Enrollment e
JOIN Student s
ON e.student_id = s.student_id
JOIN Course c
ON e.course_id = c.course_id
WHERE e.marks = (
    SELECT MAX(e2.marks)
    FROM Enrollment e2
    WHERE e2.course_id = e.course_id
);
```

Output:-

	course_name	student_name	marks
▶	Python	Rahul	78
	DBMS	Priya	92
	Data Structures	Priya	88
	Digital Electronics	Aman	35
	Thermodynamics	Neha	67
	Machine Learning	Simran	95

3.22 Departments where no student is registered

```
SELECT d.department_name
FROM Department d
LEFT JOIN Student s
ON d.department_id = s.department_id
WHERE s.student_id IS NULL;
```

Output:-

	department_name
▶	Civil
	Mathematics

3.23 Courses with no enrolment

```
SELECT c.course_name
FROM Course c
LEFT JOIN Enrollment e
ON c.course_id = e.course_id
WHERE e.enrollment_id IS NULL;
```

Output:-

	course_name
▶	Calculus

3.24 Total students in each department

```
SELECT
    d.department_name,
    COUNT(s.student_id) AS total_students
FROM Department d
LEFT JOIN Student s
ON d.department_id = s.department_id
GROUP BY d.department_id, d.department_name;
```

Output:-

Result Grid	Filter Rows:
department_name	total_students
Computer Science	3
Electronics	1
Mechanical	1
Civil	0
Artificial Intelligence	1
Mathematics	0

3.25 Complete student report with Pass/Fail

```

SELECT
    s.student_id,
    s.student_name,
    d.department_name AS department,
    c.course_name AS course,
    e.marks,
    CASE
        WHEN e.marks >= 40 THEN 'Pass'
        ELSE 'Fail'
    END AS result_status
FROM Student s
JOIN Department d
ON s.department_id = d.department_id
JOIN Enrollment e
ON s.student_id = e.student_id
JOIN Course c
ON e.course_id = c.course_id;

```

Output:-

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	student_id	student_name	department	course	marks	result_status
▶	101	Rahul	Computer Science	DBMS	85	Pass
	101	Rahul	Computer Science	Python	78	Pass
	102	Priya	Computer Science	DBMS	92	Pass
	102	Priya	Computer Science	Data Structures	88	Pass
	103	Aman	Electronics	Digital Electronics	35	Fail
	104	Neha	Mechanical	Thermodynamics	67	Pass
	106	Simran	Artificial Intelligence	Machine Learning	95	Pass
	107	Arjun	Computer Science	DBMS	72	Pass
	107	Arjun	Computer Science	Python	39	Fail

Self Join

3.26 Employee with manager name

```
SELECT
    e.emp_name AS employee,
    m.emp_name AS manager
FROM Employee e
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;
```

Output:-

	employee	manager
▶	Raj	NULL
	Amit	Raj
	Neha	Raj
	Simran	Amit
	Karan	Amit

Challenge Questions

3.27 Departments having more than 5 students

```
SELECT
    d.department_name,
    COUNT(s.student_id) AS total_students
FROM Department d
JOIN Student s
ON d.department_id = s.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(s.student_id) > 5;
```

Output:-

	department_name	total_students

3.28 Second highest marks using JOIN

```
SELECT DISTINCT e1.marks AS second_highest_marks
FROM Enrollment e1
JOIN Enrollment e2
ON e1.marks < e2.marks
ORDER BY e1.marks DESC
LIMIT 1;
```

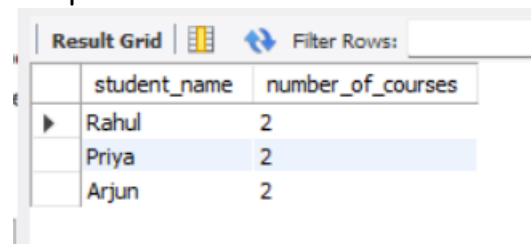
Output:-

	second_highest_marks
▶	92

3.29 Students enrolled in more than one course

```
SELECT
    s.student_name,
    COUNT(e.course_id) AS number_of_courses
FROM Student s
JOIN Enrollment e
ON s.student_id = e.student_id
GROUP BY s.student_id, s.student_name
HAVING COUNT(e.course_id) > 1;
```

Output:-



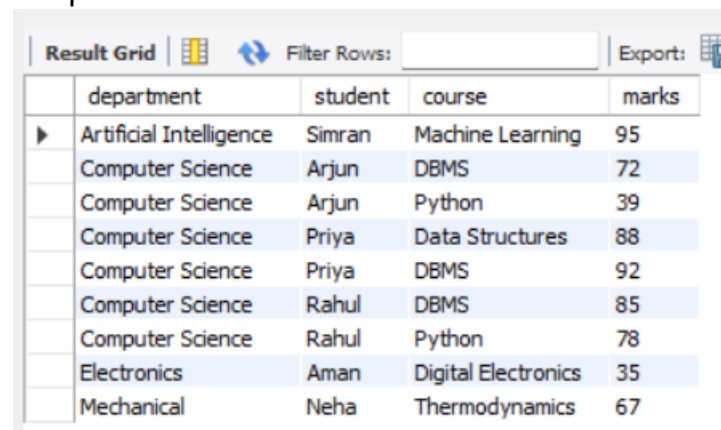
The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains three columns: 'student_name' and 'number_of_courses'. There are three rows of data, all with a value of 2 in the 'number_of_courses' column.

student_name	number_of_courses
Rahul	2
Priya	2
Arjun	2

3.30 Complete college report

```
SELECT
    d.department_name AS department,
    s.student_name AS student,
    c.course_name AS course,
    e.marks
FROM Department d
JOIN Student s
ON d.department_id = s.department_id
JOIN Enrollment e
ON s.student_id = e.student_id
JOIN Course c
ON e.course_id = c.course_id
ORDER BY d.department_name, s.student_name, c.course_name;
```

Output:-



The screenshot shows a 'Result Grid' window with an 'Export' button. The grid contains five columns: 'department', 'student', 'course', and 'marks'. There are ten rows of data, sorted by department, then student, then course.

department	student	course	marks
Artificial Intelligence	Simran	Machine Learning	95
Computer Science	Arjun	DBMS	72
Computer Science	Arjun	Python	39
Computer Science	Priya	Data Structures	88
Computer Science	Priya	DBMS	92
Computer Science	Rahul	DBMS	85
Computer Science	Rahul	Python	78
Electronics	Aman	Digital Electronics	35
Mechanical	Neha	Thermodynamics	67